

Performance Analysis of Open Channel Sensing and Impulse Noise Filtering Techniques in Noisy Communication Systems

Fabio Salgado, Jasper Mu-Chun Hsu, Glenn Zhang

May 6, 2026

1 Introduction

In many communication systems, reliable transmission begins with open channel detection, where an unoccupied channel with minimal noise is selected for communication. Multiple methods exist for identifying such channels, but the effectiveness of each approach depends on system conditions such as the number of available channels and the level of background noise.

This project investigates and compares four channel detection methods: mean squared energy (MSE), adaptive thresholding, Welch power spectral density (PSD), and autocorrelation-based detection (AC) in terms of both computational efficiency (speed) and detection accuracy (success rate). After selecting the optimal channel, a speech signal is transmitted while being intentionally corrupted by an impulsive noise component of varying amplitude (height) and duration (width), simulating real-world sudden interference.

At the receiver, several filtering techniques are applied to mitigate the effects of this impulse noise, including smart median filtering, adaptive least mean squares filtering, threshold-based clipping, and wavelet denoising. The performance of these methods is evaluated based on their ability to remove impulse noise while preserving the integrity of the original speech signal. Overall system performance is assessed using signal-to-noise ratio (SNR) improvement and qualitative audio comparison. The project aims to determine which channel detection and filtering methods perform best under varying conditions, including channel count, background noise level, impulse amplitude, and impulse duration.

2 Project Description

A simulation of the performances of the channel detection and filtering methods is done via the use of MATLAB.

For open channel detection, the simulation considers two variables: channel count, and background noise. To simulate a channel, a vector of samples suffices. The number of channels will be made to sweep to increasingly large values. For each channel, a base level of white Gaussian noise is added. The strength of the noise is also swept to increasingly large values. A random number of these channels will have an audio signal file added to the noise, simulating a transmitting channel. Each occupied channel is flagged, allowing for knowledge of which channels are actually occupied.

The methods of mean squared energy, adaptive thresholding, Welch PSD, and autocorrelation-based detection are then applied to all channels to find the clearest channel for transmission. These methods are implemented to flag occupied channels. A comparison is then made between each method's execution speed and success rate in accordance with channel count and background noise level. Specifically for success rate, a region of convergence approach is used,

considering multiple success statistics such as true positives (matches), false positives (flagged match but channel open), false negatives (flagged open but truly occupied), and true negatives (correct open detection) across all thresholds. For execution time, a simple `tic` and `toc` function before and after each method records speed.

Upon finding the cleanest open channel, a speech signal is imported and injected with an impulse of varying height and width at a random location. The SNR of this signal is computed for reference. The impulse height and width are independently swept to higher values. To simulate the receiver end, the impulse is filtered out using the methods of clipping, smart median filtering, wavelet denoising, and adaptive LMS filtering. Comparisons between the methods are made by examining the differences between the newly filtered signal’s SNR and the original transmitted signal. Conclusions can then be made for the best use case of each method depending on impulse height and width.

3 Developed Algorithms and Methods

3.1 Channel Simulation

Channels are simulated by first populating all N channels with white Gaussian noise at a specified input power level. Noise levels are expressed in dB and evaluated across the set $\{-5, 0, 5, 8, 10, 13, 15, 20, 30\}$ dB, with channel counts of $\{10, 50, 100, 500\}$ tested independently for each noise level. Note channels are of sample length 1024 and sample rate of 1000 Hz. Occupied channels are injected with a sinusoidal carrier of the form:

$$\sin\left(2\pi \cdot \frac{f_{\text{Hz}}}{F_s} \cdot t + \phi\right) \quad (1)$$

where f_{Hz} is drawn uniformly at random from 5–250 Hz (well below the Nyquist limit of 500 Hz), and ϕ is a random phase drawn uniformly from $[0, 2\pi]$. The carrier is normalized and scaled to a fixed signal power of 0.5.

To simulate realistic transmission behavior, a burst model is applied. With a 90% probability at each step, the transmitter is active for a randomly chosen duration of 200–400 samples; otherwise, it is silent for 5–20 samples. The carrier signal is multiplied element-wise by this burst envelope before being added to the noise floor. A ground truth vector records a value of 1 for each occupied channel and 0 otherwise, providing the reference labels used to compute all accuracy metrics.

3.2 Detection Methods

To evaluate each detection method, the full range of decision thresholds is swept to construct a Receiver Operating Characteristic (ROC) curve, with the Area Under the Curve (AUC) serving as the primary accuracy metric.

- **Energy (MSE) Detector:** The per-channel energy score is computed as the mean squared value of each channel’s samples. A threshold is swept across the range of observed scores to generate the ROC curve.
- **Adaptive Threshold Detector:** This method uses the same mean-squared energy score but normalizes each channel relative to the population by dividing by the median energy. This adaptive normalization reduces sensitivity to absolute power levels, making the detector more robust across varying noise floors.
- **Welch PSD Detector:** This method exploits the spectral distribution of channel energy. Occupied channels concentrate energy at the discrete frequency of their sinusoidal carrier. Welch’s power spectral density estimate is applied to each channel, and a spectral concentration measure is used as the occupancy score.

- **Autocorrelation (AC) Detector:** This method exploits the structure present in periodic carrier signals. White Gaussian noise exhibits near-zero autocorrelation at non-zero lags, whereas a sinusoidal signal produces strong, repeating autocorrelation peaks.

3.3 Impulse Noise Filtering Methods

Four digital signal processing filters were developed to mitigate the injected impulse noise at the receiver end. Recognizing that applying a global filter to an entire audio file distorts the underlying clean speech, the methods were largely designed as targeted *anomaly detectors*.

- **Threshold-Based Clipping:** This acts as a hard limiter. Any amplitude exceeding a normalized threshold of ± 1.01 is clipped. This method is instantaneous but leaves harmonic distortion at the clipping sites.
- **Smart Median (Hampel) Filter:** A 101-sample sliding median is calculated across the signal. If a specific sample deviates from the local median by an absolute value greater than 2.01, it is flagged as an impulse and replaced by the median value. Clean samples are left untouched.
- **Wavelet Denoising:** The signal is decomposed using 'sym4' wavelets. Soft thresholding is applied to remove anomalous high-frequency components associated with sudden spikes before reconstructing the audio.
- **Adaptive LMS Filter:** A 16th-order Adaptive Least Mean Squares (LMS) filter acts as a rolling predictor. It uses past samples to predict the next value. If the true sample deviates from the prediction by an error > 2.0 , it is classified as a spike and replaced by the safe prediction, protecting the filter weights from corruption.

4 Simulation Examples

4.1 Detection Accuracy

After executing the Monte Carlo trials, several clear trends emerge. At low noise levels (-5 to 5 dB), all four methods perform well, each achieving an AUC above 0.90. Conversely, at high noise levels (above 20 dB), all methods degrade toward an AUC near 0.5, equivalent to random guessing.

The most informative range is the intermediate SNR region (roughly 5–15 dB), where a clear separation between methods emerges (see Figure 1). The MSE and adaptive threshold detectors perform similarly to one another, which is expected given that both are energy-based approaches. These two methods sit at the lower end of the accuracy rankings in this region.

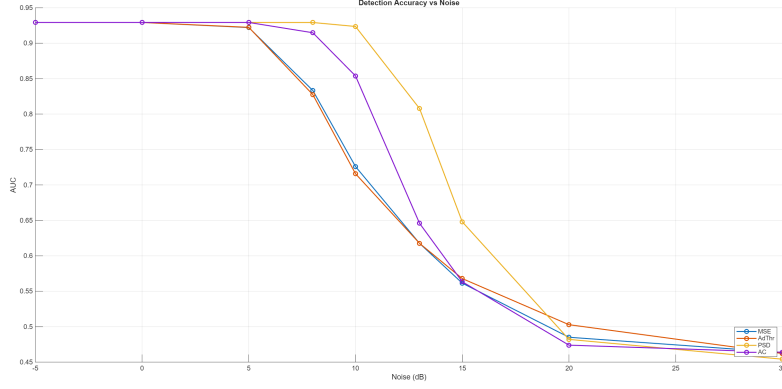


Figure 1: Area Under the Curve (AUC) values averaged from all channel counts plotted against noise for each method.

The autocorrelation detector performs notably better than the energy-based methods, benefiting from its ability to detect the periodic structure of the carrier rather than relying solely on power level. However, the Welch PSD detector consistently achieves the highest AUC in this SNR range. The rank ordering of methods is consistent across all tested channel counts, as visualized by the performance surfaces (see Figure 2).

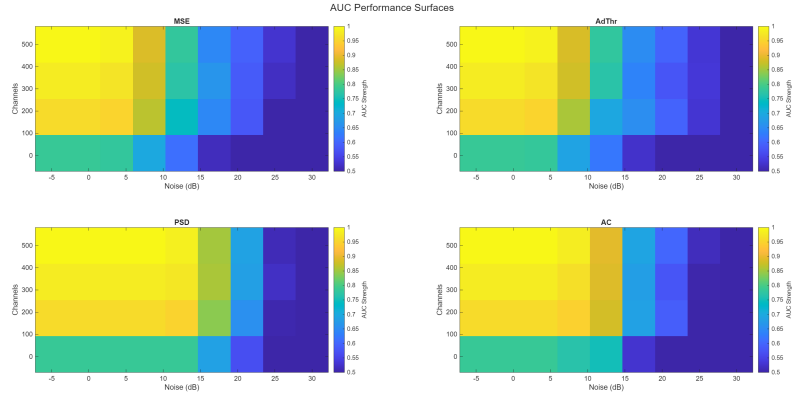


Figure 2: A heatmap representation of Noise vs. Channel Count vs. AUC strength for each method.

4.2 Execution Time

Regarding computational performance, the MSE and adaptive threshold detectors are the fastest, operating with complexity $\mathcal{O}(N \times T)$. The autocorrelation detector, implemented using FFT-based convolution, operates at $\mathcal{O}(N \times T \log T)$, resulting in moderately higher execution times. The Welch PSD detector incurs a massive constant factor due to repeated FFT computations across multiple overlapping windows, placing it clearly last in execution time performance (see Figure 3).

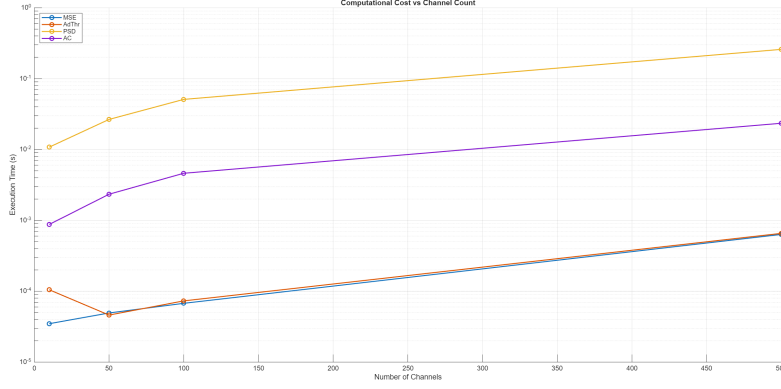


Figure 3: Number of channels vs. Execution time averaged from all noise levels for each method.

4.3 Filtering Effectiveness

To evaluate filtering success, an impulse amplitude multiplier of $10\times$ was applied to the transmitted audio, and the impulse width was swept from 1 to 50 samples. The resulting Signal-to-Noise Ratios are shown in Figure 4.

As expected, the unfiltered (corrupted) baseline SNR rapidly decays from ~ 16 dB down to nearly 0 dB as the impulse width increases. The **Adaptive LMS filter** and the **Smart Median filter** proved remarkably robust. Because they act as targeted anomaly detectors, they successfully replaced the spikes without distorting the underlying speech, yielding final SNRs ranging between 30 dB and 65 dB.

The **Clipping** filter provided moderate SNR improvements (20 dB to 35 dB); while it successfully prevented massive amplitude peaks, the flattened "clipped" tops introduced localized distortion. Interestingly, the **Wavelet** denoising method performed poorly under these specific conditions, matching or slightly underperforming the corrupted baseline. This suggests that the extreme magnitude ($10\times$) and broad frequency content of the impulse severely overwhelmed the wavelet soft-thresholding algorithms, blurring the noise into adjacent time domains rather than removing it.

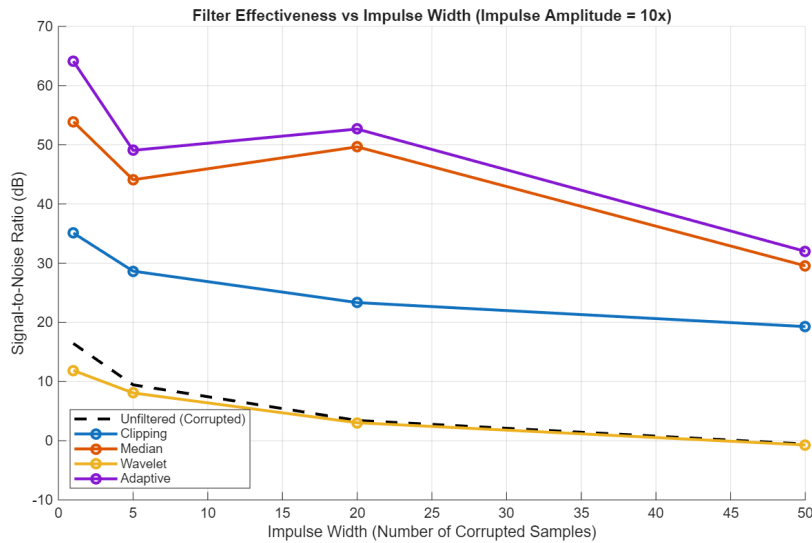


Figure 4: Filter Effectiveness (SNR) vs. Impulse Width for an Impulse Amplitude of $10\times$.

5 Conclusion

Several practical conclusions can be drawn from the open-channel detection results. At low noise levels, all four methods perform comparably in terms of accuracy, shifting the deciding factor to computational cost. In this case, the MSE or adaptive threshold detectors are the recommended choice, as they preserve detection accuracy while running significantly faster than the alternatives. In intermediate SNR ranges (5–15 dB), if detection accuracy is the top priority, the Welch PSD detector is the strongest performer. The autocorrelation detector offers an excellent middle ground, outperforming energy-based methods in accuracy while remaining considerably faster than Welch PSD.

Regarding receiver-side DSP, managing impulse noise requires targeted algorithms rather than global filters. Hard clipping is computationally cheap but leaves harmonic distortion. The Smart Median and Adaptive LMS predictor algorithms vastly outperformed traditional methods by identifying the localized anomaly and replacing it without touching the uncorrupted speech, resulting in massive SNR gains.

6 Limitations and Broader Considerations

Several factors outside the scope of this model are worth noting. The autocorrelation detector relies on the presence of repeating structure in the transmitted signal. If real-world carriers exhibit less periodicity, autocorrelation performance would degrade. The Welch PSD detector is highly sensitive to frequency resolution; in scenarios with shorter effective signal durations, the spectral energy becomes smeared across frequency bins, reducing its detection advantage. Finally, while the Adaptive LMS filter performed exceptionally well on isolated impulses, tuning the learning rate (μ) and prediction threshold requires prior knowledge of the underlying speech dynamics, which may be difficult to generalize in rapidly changing communication environments.

References

- [1] Olumide, S. (2026, February 18). *A complete guide to ROC curves and AUC*. Statology. <https://www.statology.org/a-complete-guide-to-roc-curves-and-auc/>
- [2] Solomon, O. M. (1992, December). *PSD Computations Using Welch’s Method*. Sandia Report SAND 91-1533 UC-706. <https://www.osti.gov/servlets/purl/5688766>

A MATLAB Source Code

A.1 main_simulation.m

```
1 %% Master Simulation Script
2 clc; clear; close all;
3
4 % Add the subfolder to the MATLAB search path
5 addpath('functions');
6
7 %% Setup
8 N_channels_array = [10, 50, 100, 500];
9 noise_snr_array = [-5, 0, 5, 8, 10, 13, 15, 20, 30];
10 impulse_heights = [2, 5, 10, 20];
11 impulse_widths = [1, 5, 20, 50];
12
13 detection_methods = {'MSE', 'AdThr', 'PSD', 'AC'};
14 filter_methods = {'Clipping', 'Median', 'Wavelet', 'Adaptive'};
15
16 detection_results = struct();
17 filtering_results = struct();
18
19 % Load Audio
20 load handel.mat;
21 clean_audio = y;
22 clean_audio = clean_audio / max(abs(clean_audio));
23
24 %% Environment (Phase 1 & 2)
25 K = 20; % number of iterations
26
27 for n_idx = 1:length(N_channels_array)
28     N = N_channels_array(n_idx);
29     for noise_idx = 1:length(noise_snr_array)
30         noise_snr = noise_snr_array(noise_idx);
31         for det_idx = 1:length(detection_methods)
32             method = detection_methods{det_idx};
33             auc_runs = zeros(K,1);
34             time_runs = zeros(K,1);
35
36             for k = 1:K
37                 [Channel_Matrix, Ground_Truth] = phase1_generate_env(N,
38 noise_snr, 0.3);
39                 [AUC, Exec_Time] = phase2_detectors(Channel_Matrix,
40 Ground_Truth, method);
41                 auc_runs(k) = AUC;
42                 time_runs(k) = Exec_Time;
43             end
44
45             detection_results(n_idx, noise_idx, det_idx).Time = mean(time_runs)
46 ;
47             detection_results(n_idx, noise_idx, det_idx).AUC = mean(auc_runs);
48         end
49     end
50 end
51
52 if ~exist('data', 'dir'), mkdir('data'); end
53 save('data/Detection_Data.mat', 'detection_results', 'N_channels_array', '
54 noise_snr_array');
55
56 %% Transmission (Phase 3 & 4)
57 for h_idx = 1:length(impulse_heights)
58     height = impulse_heights(h_idx);
59     for w_idx = 1:length(impulse_widths)
```

```

56         width = impulse_widths(w_idx);
57
58         [corrupted_audio, baseline_SNR] = phase3_corrupt(clean_audio, height,
59 width);
60
61         for filt_idx = 1:length(filter_methods)
62             f_method = filter_methods{filt_idx};
63             [cleaned_audio, new_SNR] = phase4_filters(corrupted_audio,
64 clean_audio, f_method);
65
66             filtering_results(h_idx, w_idx, filt_idx).BaselineSNR =
67 baseline_SNR;
68             filtering_results(h_idx, w_idx, filt_idx).NewSNR = new_SNR;
69         end
70     end
71 end
72
73 save('data/Filtering_Data.mat', 'filtering_results', 'impulse_heights', '
74 impulse_widths');

```

A.2 phase1_generate_env.m

```

1 function [Channel_Matrix, Ground_Truth] = phase1_generate_env(N, noise_level_dB
2 , occupancy_rate)
3     % Inputs:
4     %     N - Number of channels
5     %     noise_level_dB - Background WGN level
6     %     occupancy_rate - Representing % of occupied channels
7
8     % Define number of time samples per channel
9     T = 1024; % (you can tune this)
10
11     Fs = 1000; % Sample rate in Hz
12
13     % Initialize empty Channel_Matrix
14     Channel_Matrix = zeros(N, T);
15
16     % Initialize Ground_Truth array of zeros
17     Ground_Truth = zeros(N, 1);
18
19     % calculate linear noise and add it to all channels
20     noise_power = 10^(noise_level_dB / 10);
21     Channel_Matrix = sqrt(noise_power) * randn(N, T);
22     signal_power = 0.5; % fixed
23
24     % Randomly select channels based on occupancy_rate
25     numOccupied = round(N * occupancy_rate); % Calculate number of occupied
26 channels
27     occupiedChannels = randperm(N, numOccupied); % Randomly select occupied
28 channels
29
30     % - Change Ground_Truth to 1 for these channels
31     Ground_Truth(occupiedChannels) = 1;
32
33     % - Inject simulated semi random transmission signals into these rows
34     t = (0:T-1);
35
36     for i = 1:length(occupiedChannels)
37         ch = occupiedChannels(i);
38
39         % Frequency: true Hz, stays below Nyquist/2 for clean spectrum

```

```

38     f_Hz = randi([5, floor(Fs/4)]); % 5 250 Hz (Nyquist = 500 Hz)
39
40     % random phase
41     phi = 2*pi*rand;
42
43     % Carrier signal
44     tx_signal = sin(2*pi * (f_Hz/Fs) * t + phi);
45
46     % normalize signal power to control SNR
47     tx_signal = tx_signal / rms(tx_signal);
48
49     % scale to match SNR (signal is fixed to 1)
50     tx_signal = sqrt(signal_power) * tx_signal;
51
52     % simulate bursts in transmission
53     active_prob = 0.9;
54     burst = zeros(1, T);
55     t_idx = 1;
56     while t_idx <= T
57         if rand < active_prob
58             on_len = randi([200, 400]);
59             burst(t_idx : min(t_idx + on_len - 1, T)) = 1;
60             t_idx = t_idx + on_len;
61         else
62             off_len = randi([5, 20]);
63             t_idx = t_idx + off_len;
64         end
65     end
66
67     % populate channel with signal
68     Channel_Matrix(ch, :) = Channel_Matrix(ch, :) + tx_signal .* burst;
69
70 end
71
72 end

```

A.3 phase2_detectors.m

```

1 function [AUC, Exec_Time] = phase2_detectors(Channel_Matrix, Ground_Truth,
2     Method_String)
3 % Returns:
4 % AUC      - ROC performance (area under curve )
5 % Exec_Time - feature extraction runtime ONLY
6
7 % Select Method
8 tic;
9
10 switch Method_String
11     case 'MSE'
12         score = MSE_score(Channel_Matrix);
13
14     case 'AdThr'
15         score = AdThr_score(Channel_Matrix);
16
17     case 'PSD'
18         score = PSD_score(Channel_Matrix);
19
20     case 'AC'
21         score = AC_score(Channel_Matrix);
22
23     otherwise

```

```

24         error('Unknown Method_String');
25     end
26
27     Exec_Time = toc;    % detector runtime
28
29     % ROC accuracy results
30     [AUC] = ROC(score, Ground_Truth);
31
32     end
33
34
35     % compute score for ROC with MSE method
36     function score = MSE_score(X)
37         score = mean(X.^2, 2);
38     end
39
40     % compute score for ROC with Adaptive Threshold method
41     function score = AdThr_score(X)
42         energy = mean(X.^2, 2);
43         score = energy ./ (median(energy) + eps);
44     end
45
46     % compute score for ROC with PSD method
47     function score = PSD_score(X)
48
49         N = size(X,1);
50         score = zeros(N,1);
51
52         % Persistent Welch parameters (created once)
53         persistent win noverlap nfft Fs
54         if isempty(win)
55             Fs = 1000;           % sample rate
56             win = hamming(512);  % longer window so fewer segments
57             noverlap = 256;      % moderate overlap
58             nfft = 512;          % balanced resolution/speed
59         end
60
61         for i = 1:N
62             x = single(X(i,:));    % single precision = faster FFT
63
64             % Welch PSD
65             Pxx = pwelch(x, win, noverlap, nfft, Fs);
66
67             % Feature: peak PSD
68             score(i) = max(Pxx);
69         end
70     end
71
72
73     % compute score for ROC with AC method
74     function score = AC_score(X)
75
76     N = size(X,1);
77     score = zeros(N,1);
78
79     for i = 1:N
80         x = X(i,:);
81
82         % see how similiar the signal is to a shifted copy of itself
83         % noisy signals will not look similar
84         r = xcorr(x, 'coeff');
85
86         % mid is always a match, remove it

```

```

87     mid = ceil(length(r)/2);
88     r(mid) = 0;
89
90     score(i) = max(abs(r));
91 end
92
93 end
94
95 % compute accuracy variables using method score
96 function [AUC] = ROC(score, GT)
97
98 % choose ROC thresholds
99 thresholds = prctile(score, linspace(0,100,200));
100
101 % instantiate false alarm rate and detection rate
102 Pfa = zeros(length(thresholds),1);
103 Pd = zeros(length(thresholds),1);
104
105 for i = 1:length(thresholds)
106
107     thr = thresholds(i);
108
109     % determine occupation
110     preds = score > thr;
111
112     % calculate accuracy stats from actual known results
113     TP = sum(preds == 1 & GT == 1); % true positives
114     FP = sum(preds == 1 & GT == 0); % false positives
115     FN = sum(preds == 0 & GT == 1); % false negatives
116     TN = sum(preds == 0 & GT == 0); % true negatives
117
118     % calculate false alarm rate and detection rate
119     Pd(i) = TP / (TP + FN + eps);
120     Pfa(i) = FP / (FP + TN + eps);
121
122 end
123
124 % sort the rates and calculate area under curve (for graph)
125 [Pfa_sorted, idx] = sort(Pfa);
126 Pd_sorted = Pd(idx);
127
128 AUC = trapz(Pfa_sorted, Pd_sorted);
129
130 end

```

A.4 phase3_corrupt.m

```

1 function [corrupted_audio, baseline_SNR] = phase3_corrupt(audio, impulse_height
, impulse_width)
2     % Inputs:
3     %     audio          - Column vector of original clean audio
4     %     impulse_height - Amplitude multiplier of the spike
5     %     impulse_width  - Number of samples the spike lasts
6
7     corrupted_audio = audio;
8     N_samples = length(audio);
9
10    % Generate a random index within the bounds of the audio length
11    start_idx = randi([1, max(1, N_samples - impulse_width + 1)]);
12    end_idx = min(start_idx + impulse_width - 1, N_samples);
13
14    % Apply the impulse_height to the audio starting at the random index

```

```

15     spike_amplitude = impulse_height * max(abs(audio));
16
17     % Create an alternating sign burst to simulate broadband impulse scatter
18     burst_signs = sign(randn(end_idx - start_idx + 1, 1));
19     corrupted_audio(start_idx:end_idx) = corrupted_audio(start_idx:end_idx) + (
    spike_amplitude * burst_signs);
20
21     % Calculate Baseline SNR
22     signal_power = var(audio);
23     noise_power = var(corrupted_audio - audio);
24
25     if noise_power == 0
26         baseline_SNR = Inf;
27     else
28         baseline_SNR = 10 * log10(signal_power / noise_power);
29     end
30 end

```

A.5 phase4_filters.m

```

1 function [cleaned_audio, new_SNR] = phase4_filters(corrupted_audio, clean_audio
, Filter_Type)
2     switch Filter_Type
3         case 'Clipping'
4             cleaned_audio = apply_clipping(corrupted_audio);
5         case 'Median'
6             cleaned_audio = apply_median(corrupted_audio);
7         case 'Wavelet'
8             cleaned_audio = apply_wavelet(corrupted_audio);
9         case 'Adaptive'
10            cleaned_audio = apply_adaptive(corrupted_audio);
11        otherwise
12            error('Unknown Filter_Type string provided.');

```

```

42     catch
43         cleaned_audio = smoothdata(X, 'movmean', 5);
44     end
45 end
46
47 function cleaned_audio = apply_adaptive(X)
48     order = 16; mu = 0.1; w = zeros(order, 1);
49     cleaned_audio = zeros(size(X));
50     x_memory = zeros(order, 1);
51
52     for n = 1:length(X)
53         y_hat = w' * x_memory;
54         e = X(n) - y_hat;
55
56         if abs(e) > 2.0
57             safe_sample = y_hat;
58         else
59             w = w + (mu) * e * x_memory;
60             safe_sample = X(n);
61         end
62         cleaned_audio(n) = safe_sample;
63         x_memory = [safe_sample; x_memory(1:end-1)];
64     end
65 end

```

A.6 visualization

```

1 %% Plotting Script
2 % Load the generated data from main_simulation.m
3 load('data/Detection_Data.mat');
4 load('data/Filtering_Data.mat');
5
6 detection_methods = {'MSE', 'AdThr', 'PSD', 'AC'};
7 filter_methods    = {'Clipping', 'Median', 'Wavelet', 'Adaptive'};
8
9 %% 1. Sensing Speed v Channel Count
10 figure; hold on;
11 colors = lines(length(detection_methods));
12
13 for det_idx = 1:length(detection_methods)
14     time_curve = zeros(1, length(N_channels_array));
15     for n_idx = 1:length(N_channels_array)
16         vals = zeros(length(noise_snr_array),1);
17         for noise_idx = 1:length(noise_snr_array)
18             vals(noise_idx) = detection_results(n_idx,noise_idx,det_idx).Time;
19         end
20         time_curve(n_idx) = mean(vals);
21     end
22     plot(N_channels_array, time_curve, '-o', 'Color', colors(det_idx,:), ...
23          'LineWidth', 1.5, 'DisplayName', detection_methods{det_idx});
24 end
25
26 set(gca, 'YScale', 'log')
27 xlabel('Number of Channels');
28 ylabel('Execution Time (s)');
29 title('Computational Cost vs Channel Count');
30 legend('Location', 'northwest');
31 grid on;
32
33 %% 2. Sensing Accuracy v Background Noise
34 figure; hold on;
35 colors = lines(length(detection_methods));

```

```

36
37 for det_idx = 1:length(detection_methods)
38     auc_curve = zeros(1, length(noise_snr_array));
39     for noise_idx = 1:length(noise_snr_array)
40         % average over channel dimension
41         vals = zeros(length(N_channels_array),1);
42         for n_idx = 1:length(N_channels_array)
43             vals(n_idx) = detection_results(n_idx,noise_idx,det_idx).AUC;
44         end
45         auc_curve(noise_idx) = mean(vals);
46     end
47     plot(noise_snr_array, auc_curve, '-o', 'Color', colors(det_idx,:), ...
48         'LineWidth', 1.5, 'DisplayName', detection_methods{det_idx});
49 end
50
51 xlabel('Noise (dB)');
52 ylabel('AUC');
53 title('Detection Accuracy vs Noise');
54 legend('Location', 'southeast');
55 grid on;
56
57 %% 3. Channel, Method, and Noise 3D plot
58 figure;
59 for det_idx = 1:length(detection_methods)
60     Z = zeros(length(N_channels_array), length(noise_snr_array));
61     for n_idx = 1:length(N_channels_array)
62         for noise_idx = 1:length(noise_snr_array)
63             Z(n_idx, noise_idx) = detection_results(n_idx,noise_idx,det_idx).
64             AUC;
65         end
66     end
67     subplot(2,2,det_idx);
68     imagesc(noise_snr_array, N_channels_array, Z);
69     set(gca,'YDir','normal');
70     title(detection_methods{det_idx});
71     xlabel('Noise (dB)');
72     ylabel('Channels');
73     caxis([0.5 1]);
74     h = colorbar;
75     ylabel(h, 'AUC Strength');
76 end
77 sgtitle('AUC Performance Surfaces');
78
79 %% 4. Filter Effectiveness (SNR Improvement)
80 figure('Name', 'SNR Improvement vs Impulse Width'); hold on; grid on;
81 colors_filt = lines(length(filter_methods));
82
83 % Choose a fixed Impulse Height to graph (e.g., index 3 out of 4 -> 10x height)
84 fixed_h_idx = 3;
85 selected_height = impulse_heights(fixed_h_idx);
86
87 % Plot the Corrupted (Baseline) SNR
88 baseline_snr_curve = zeros(1, length(impulse_widths));
89 for w_idx = 1:length(impulse_widths)
90     baseline_snr_curve(w_idx) = filtering_results(fixed_h_idx, w_idx, 1).
91     BaselineSNR;
92 end
93 plot(impulse_widths, baseline_snr_curve, '--k', 'LineWidth', 2, 'DisplayName',
94     'Unfiltered (Corrupted)');
95
96 % Plot the Cleaned SNR for each filter method
97 for filt_idx = 1:length(filter_methods)
98     filtered_snr_curve = zeros(1, length(impulse_widths));

```

```

96     for w_idx = 1:length(impulse_widths)
97         filtered_snr_curve(w_idx) = filtering_results(fixed_h_idx, w_idx,
100         filt_idx).NewSNR;
101     end
102
103     plot(impulse_widths, filtered_snr_curve, '-o', ...
104         'Color', colors_filt(filt_idx,:), ...
105         'LineWidth', 2, ...
106         'MarkerSize', 6, ...
107         'DisplayName', filter_methods{filt_idx});
108 end
109
110 xlabel('Impulse Width (Number of Corrupted Samples)');
111 ylabel('Signal-to-Noise Ratio (dB)');
112 title(sprintf('Filter Effectiveness vs Impulse Width (Impulse Amplitude = %dx)',
113     , selected_height));
114 legend('Location', 'southwest');

```